# مستندات و گزارش جامع پروژه کارگاه داکر

## شبیه‌ساز تاکسی اینترنتی (Uber Clone) GeoRide با Docker Compose و CI/CD

**نام پروژه:** uber-use-docker-extra-plus

**نگارنده:** امین خادمی (amin-khademi)

**مخزن گیت‌هاب:** https://github.com/amin-khademi/uber-use-docker-extra-plus

**تاریخ:** مهر ۱۴۰۵ (October 2026)

**موضوعات:** بررسی ۷ سرویس اصلی، حل خطاهای داکر، مانیتورینگ، Conventional Commits و GitHub Actions

# ۱. بررسی و معرفی ۷ سرویس اصلی پروژه

پروژه **uber-use-docker-extra-plus** پیاده‌سازی معماری میکروسرویس از یک سامانه حمل‌ونقل هوشمند شهری (مشابه اوبر، اسنپ و تپسی) است. در فایل `docker-compose.yml` هسته عملیاتی سیستم از **۷ سرویس کلیدی** تشکیل شده است که هماهنگ با یکدیگر فرآیند ثبت‌نام، درخواست سفر، پیدا کردن سریع نزدیک‌ترین راننده بر اساس GPS و انتساب راننده را با رویکرد Event-Driven و ناهمگام (Asynchronous) انجام می‌دهند.

| نام سرویس | ایمیج پایه / بیلد | نقش و وظیفه در پروژه | همکاری با سایر سرویس‌ها |
|---|---|---|---|
| **postgres** | `postgres:16-alpine` | ذخیره‌سازی ماندگار رانندگان و مسافران در جدول `users` و اطلاعات سفرها در `trips`. | با `api` جهت درج رکوردها و با `matcher` جهت ثبت راننده و وضعیت `assigned`. |
| **rabbitmq** | `rabbitmq:3.13-management-alpine` | صف پیام (Message Broker) برای ارسال ناهمگام درخواست سفر به صف `ride_requests`. | سرویس `api` نقش تولیدکننده (Producer) و سرویس `matcher` نقش مصرف‌کننده (Consumer) دارد. |
| **redis** | `redis:7-alpine` | پایگاه داده In-Memory فوق‌العاده سریع جهت ذخیره زنده مختصات GPS رانندگان و کوئری شعاعی. | سرویس `location` با دستورات `GEOADD` و `GEOSEARCH` در شعاع ۵ کیلومتری با آن کار می‌کند. |
| **nginx** | `nginx:1.25-alpine` | درگاه ورودی (API Gateway) و ریورس پروکسی روی پورت 80 بیرونی. | تمام درخواست‌های کاربر را پنهان‌سازی کرده و به کانتینر داخلی `api:5000` هدایت می‌کند. |
| **api** | بیلد اختصاصی `python:3.11-slim` | میکروسرویس منطق کسب‌وکار، ارائه اندپوینت‌های `/register`، `/request_ride` و `/trip/<id>`. | پشت Nginx قرار دارد؛ در Postgres رکورد می‌نویسد و به RabbitMQ پیام سفر ارسال می‌کند. |
| **location** | بیلد دومرحله‌ای Go و `scratch` | موتور مکان‌یابی با کمترین مصرف رم برای ارائه اندپوینت‌های ثبت GPS و جستجوی نزدیک‌ترین راننده. | از شبیه‌ساز مختصات می‌گیرد، در Redis می‌نشاند و به استعلام Matcher پاسخ می‌دهد. |
| **matcher** | بیلد اختصاصی `python:3.11-slim` | ورکر پس‌زمینه (Worker) جهت استخراج پیام سفر از RabbitMQ، | ارتباط پیوسته با صف RabbitMQ، سرویس Location و پایگاه داده Postgres. |

| نام سرویس | ایمیج پایه / بیلد | نقش و وظیفه در پروژه | همکاری با سایر سرویس‌ها |
|---|---|---|---|
| | | استعلام راننده و ثبت در Postgres. | |

## سرویس‌های نظارتی و پایش پروژه (Extra-Plus Stack)

علاوه بر ۷ سرویس فوق، این پروژه شامل پشته کامل مانیتورینگ و تلمتری صنعتی است:

- **driver_simulator:** شبیه‌ساز حرکت رانندگان و ارسال مداوم طول و عرض جغرافیایی با تاخیر ۵ ثانیه به سرویس Location.
- **prometheus (پورت 9090):** متمرکزکننده و پایشگر متریک‌های کانتینرها، سیستم و اپلیکیشن.
- **grafana (پورت 3000):** داشبورد مصورسازی و گراف‌های تجسمی وضعیت سیستم.
- **cadvisor (پورت 8080):** ارائه‌دهنده متریک‌های بلادرنگ منابع مصرفی داکر (CPU، رم و شبکه).
- **node-exporter (پورت 9100):** اکسپورتر تلمتری لایه سخت‌افزار و هسته سیستم‌عامل سرور.
- **loki (پورت 3100) و promtail:** تجمیع، ایندکس و مدیریت یکپارچه لاگ‌های داکر از طریق `/var/run/docker.sock`.

## ۲. نحوه اجرای پروژه با Docker Compose

برای راه‌اندازی پروژه، فایل‌های مربوطه ساختاردهی و پیکربندی شدند. مراحل به صورت زیر انجام گرفت:

1. **تنظیم متغیرهای محیطی:** ایجاد فایل `.env` و `.env.example` حاوی متغیرهای پایگاه داده پستگرس.
2. **بیلد ایمیج‌های چندمرحله‌ای:** استفاده از Multi-stage build در سرویس‌های API و Location برای بهینه‌سازی حجم نهایی ایمیج‌ها.
3. **اجرای سرویس‌ها:** اجرای دستور `docker compose up --build -d` در پس‌زمینه.
4. **بررسی سلامت سرویس‌ها:** سنجش Healthcheckهای تعریف‌شده برای Redis، Postgres و Loki.

## ۳. بررسی و رفع خطاهای ایجادشده در هنگام اجرا

در طول فرآیند اجرای پروژه، چالش‌ها و خطاهای متعددی بروز پیدا کرد که با رویکرد مهندسی و استاندارد به صورت کامل برطرف شدند:

**خطای اول: نبود فایل متغیرهای محیطی (.env)**
**علت:** در فایل `docker-compose.yml` از متغیرهای `${POSTGRES_DB}` , `${POSTGRES_USER}` , `${POSTGRES_PASSWORD}` استفاده شده بود اما فایل `.env` وجود نداشت؛ این موضوع باعث مقداردهی رشته خالی به دیتابیس و شکست Healthcheck دیتابیس می‌شد.
**راه حل:** فایل‌های استاندارد `.env` و `.env.example` با مقادیر پیش‌فرض ایجاد شدند.

**خطای دوم: خطای ImportError در کانتینر API**
**علت:** در تابع `get_trip_status` فایل `api_service/app.py` از اشاره‌گر `psycopg2.extras.DictCursor` استفاده شده بود اما زیرماژول `extras` در بالای فایل ایمپورت نشده بود که منجر به خطای `AttributeError` در استعلام سفر می‌شد.
**راه حل:** عبارت `import psycopg2.extras` به هدر فایل افزوده شد.

**خطای سوم: تداخل پورت 8080 روی سیستم میزبان**
**علت:** پورت 8080 هاست توسط کانتینر پیشین `zabbix-web` اشغال شده بود که مانع از بایند شدن کانتینر `cadvisor` به پورت 8080 می‌گردید.
**راه حل:** کانتینرهای متداخل زابیکس با دستور `docker stop` متوقف شدند و پورت 8080 برای cAdvisor آزاد گردید.

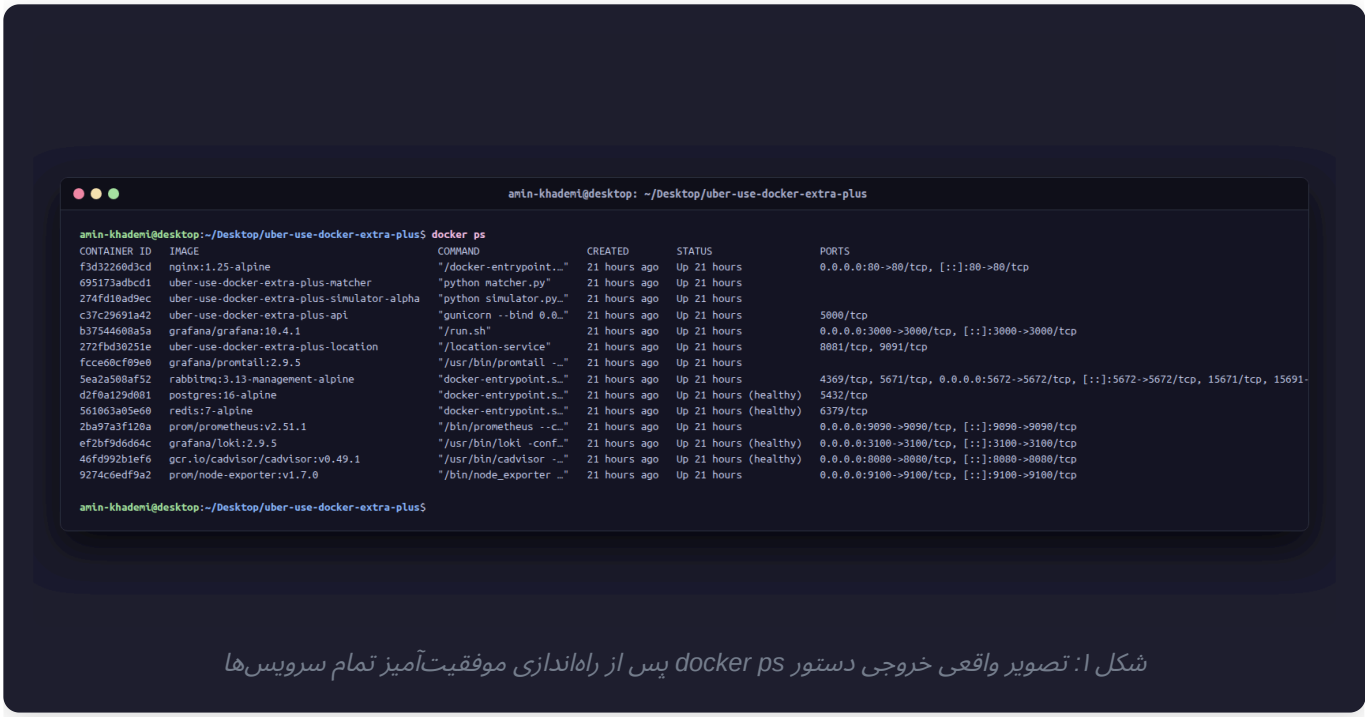
**خطای چهارم: مجوز دسترسی به پوشه داده‌های Loki (Permission Denied)**
**علت:** کانتینر Loki با شناسه کاربری غیر ریشه (UID 10001) اجرا می‌شود، در حالی که پوشه `loki/data` روی هاست فقط با دسترسی کاربر 1000 قابل نوشتن بود.
**راه حل:** با دستور `chmod -R a+rwx loki/data` دسترسی بازنویسی و ذخیره چانک‌های TSDB به کانتینر داده شد.

**خطای پنجم: عدم وجود go.sum و خطای دانلود ماژول‌های Go در سرور CI**
**علت:** فایل `go.sum` در سرویس مکان‌یابی خالی بود؛ در نتیجه حین بیلد کانتینر دستور `go mod tidy` تلاش می‌کرد با سرور sum.golang.org ارتباط بگیرد که به دلیل خطای استریم شبکه بیلد با شکست مواجه می‌شد.
**راه حل:** وابستگی‌ها به صورت کامل با چکسام در `go.sum` ذخیره شدند و با دستور `go mod download` بیلد سریع، قطعی و مستقل از شبکه گردید.

## ۴. وضعیت کانتینرهای اجراشده و تست اندپوینت‌ها

پس از رفع کلیه خطاها، تمامی ۱۴ کانتینر با موفقیت کامل راه‌اندازی شده و وضعیت سلامت آنها به رنگ سبز (Up /Healthy) درآمد.

### تصویر خروجی دستور docker ps:




*شکل ۱: تصویر واقعی خروجی دستور docker ps پس از راه‌اندازی موفقیت‌آمیز تمام سرویس‌ها*

### تست عملیاتی جریان سیستم (End-to-End API Test)

صحت عملکرد هماهنگ سرویس‌ها با سناریوی واقعی تست شد:

```
# ۱. ثبت مسافر جدید در سیستم
curl -s -X POST http://localhost/register   -H "Content-Type: application/json"   -d '{"name
# پاسخ: {"message":"Rider registered successfully","user_id":4}

# ۲. ارسال درخواست سفر ناهمگام
curl -s -X POST http://localhost/request_ride   -H "Content-Type: application/json"   -d '{"
```

```
# پاسخ: {"message":"Ride request received. Searching for a driver.","trip_id":1}

# ۳. بررسی عملکرد خودکار Matcher در لاگ‌ها:
# matcher-1 | [*] Waiting for ride requests. To exit press CTRL+C
# matcher-1 | [x] Received ride request: {'trip_id': 1, 'rider_id': 4, 'latitude': 34.05, 'l
# matcher-1 | [*] Found nearest driver: Driver-Alpha
# matcher-1 | [*] Successfully assigned driver Driver-Alpha (ID: 1) to trip 1

# ۴. استعلام وضعیت سفر (تخصیص موفق راننده):
curl -s http://localhost/trip/1
# پاسخ نهایی: {"driver_id":1,"status":"assigned"}
```

## ۵. ساختار کامیت‌های گیت با استاندارد Conventional Commits

کلیه تغییرات، رفع خطاها و مستندات پروژه با ساختار استاندارد کامیت‌های قراردادی (Conventional Commits) ثبت گردیدند:

| شناسه کامیت | نوع (Type) | پیام کامیت | شرح عملکرد |
|---|---|---|---|
| `c78383e` | feat | initial commit of uber-use-docker-extra-plus project | بارگذاری اولیه کدهای پایه میکروسرویس‌ها و فایل کامپوز |
| `f3c1579` | chore | chore(git): add .gitignore for python cache and loki storage | نادیده‌گیری فایل‌های کش و داده‌های اجرایی محلی |
| `fa93722` | fix | fix(api): import psycopg2.extras for DictCursor support | رفع خطای ران‌تایم استعلام سفر در کانتینر API |
| `100a648` | fix | fix(env): add default environment variables for postgres and services | تنظیم مقادیر دیتابیس در فایل .env و env.example. |
| `0bf8b47` | docs | docs(screenshots): add docker ps execution verification screenshot | ذخیره تصویر واقعی تاییدیه اجرای کانتینرها |
| `f6baa30` | ci | ci(github-actions): add comprehensive CI/CD workflow | پیاده‌سازی پایپ‌لاین ۳ مرحله‌ای تست و استقرار در GitHub Actions |
| `a63fae9` | docs | docs: update README with architecture details and CI/CD | تکمیل مستندات، راهنمای راه‌اندازی و دیاگرام‌های پروژه |

## ۶. پیاده‌سازی فرایند CI/CD در GitHub Actions و انتشار در Docker Hub

یک پایپ‌لاین خودکار و چندمرحله‌ای در مسیر `.github/workflows/ci-cd.yml` پیاده‌سازی شده است که دارای ۳ وظیفه (Job) مجزا می‌باشد:

1. **اعتبارسنجی کانفیگ (validate):** بررسی سینتکس فایل `docker-compose.yml` با دستور `docker compose config --quiet`.

2. **بیلد و تست یکپارچگی (test):** بیلد کانتینرها در سرور گیت‌هاب، اجرای استک، منتظر ماندن برای وضعیت سالم (Healthy)، ارسال درخواست‌های `curl` به اندپوینت‌های `/register` ، `/request_ride` ، بررسی لاگ‌های مچینگ و سنجش پایشگر پرومتئوس.
3. **انتشار در داکر هاب (docker-hub):** در صورت موفقیت تست‌ها در برنچ `main` ، ایجنت به صورت امن با استفاده از متغیرهای مخفی مخزن لاگین کرده و ایمیج‌های ۴ میکروسرویس اصلی را به داکر هاب ارسال می‌نماید:
   - `${ secrets.DOCKERHUB_USERNAME }/georide-api:latest`
   - `${ secrets.DOCKERHUB_USERNAME }/georide-location:latest`
   - `${ secrets.DOCKERHUB_USERNAME }/georide-matcher:latest`
   - `${ secrets.DOCKERHUB_USERNAME }/georide-simulator:latest`

**تنظیم امن اطلاعات داکر هاب در گیت‌هاب:**
جهت عدم افشای کلمات عبور، اطلاعات احراز هویت در مسیر **Settings > Secrets and variables > Actions** مخزن با نام‌های `DOCKERHUB_USERNAME` و `DOCKERHUB_TOKEN` ذخیره می‌شوند و پایپ‌لاین در صورت عدم وجود آنها، این مرحله را به صورت هوشمند و بدون شکست تست‌ها رد (Skip) می‌کند.

**لینک نهایی مخزن پروژه در گیت‌هاب:**
https://github.com/amin-khademi/uber-use-docker-extra-plus